

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Manipulation (Level 2)
Question Count: 5

Total Marks: 25

Code of Conduct

I kavya k(192572065) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: kavya k

Criteria	Understanding of Scenario (1)	Identification of Risks / Issues (1)	Application of Concepts (1.25)	Decision Justification (1)	Clarity (075)	TOTAL (5)
Weightage	20%	20%	25%	20%	15%	100%
Q1						
Q2						
Q3						
Q4						
Q5						

Signature of the Faculty

Total Marks Awarded:

Question 1

A financial institution plans to deploy a private cloud with strict compliance requirements. Recommend an appropriate cloud datacenter architecture and explain how virtualization and isolation mechanisms can ensure security and regulatory compliance.

Understanding of the Scenario

A financial institution handles highly sensitive data (customer PII, transaction records, account details) and must comply with regulations such as PCI-DSS, RBI/SEBI guidelines, GDPR or SOX. Because a public multi-tenant cloud carries a higher risk of data co-mingling and unauthorized access, the institution requires a dedicated, tightly controlled infrastructure with strong isolation, auditability, and data sovereignty.

Recommended Datacenter Architecture

- **Private Cloud Model:** A single-tenant private cloud hosted either on-premises or in a dedicated hosted-private cloud, so the institution retains full control over hardware, hypervisor configuration, and data location.
- **Tiered Datacenter Design:** A Tier III/IV datacenter design offering redundant power, cooling, and network paths (N+1 redundancy) to guarantee high availability for financial transactions.
- **Modular Server-Storage-Network Pods:** Segregated pods for core banking, payment processing, and analytics workloads, each isolated at the network layer.
- **Disaster Recovery Site:** A geographically separate DR datacenter with synchronous/asynchronous replication to meet Recovery Time Objective (RTO) and Recovery Point Objective (RPO) requirements mandated by regulators.

Role of Virtualization and Isolation Mechanisms

- **Type-1 (Bare-Metal) Hypervisor:** Using a bare-metal hypervisor (e.g., VMware ESXi, KVM, Hyper-V) instead of a hosted hypervisor reduces the attack surface and improves performance isolation between VMs.
- **VM and Network Isolation:** Each business unit (retail banking, payments, risk analytics) is placed in its own VM/VLAN with micro-segmentation and firewall rules, ensuring that a breach in one segment cannot propagate laterally.
- **Hardware-Assisted Security:** Use of Trusted Platform Module (TPM), secure boot, and Intel VT-x/AMD-V extensions to prevent VM escape and hypervisor tampering.
- **Encryption:** Data-at-rest encryption on virtual disks and data-in-transit encryption (TLS/IPSec) between VMs, along with key management via a Hardware Security Module (HSM).
- **Identity and Access Control:** Role-Based Access Control (RBAC) and Multi-Factor Authentication (MFA) for hypervisor and management-console access, combined with detailed audit logging for every administrative action.
- **Compliance Automation:** Continuous compliance monitoring tools and periodic vulnerability scanning/penetration testing to demonstrate adherence to PCI-DSS and other regulatory frameworks.

Decision Justification

A private, single-tenant cloud with bare-metal hypervisors and strict micro-segmentation gives the institution the isolation, auditability, and control needed for regulatory compliance, while the tiered/DR datacenter design ensures the availability and resilience expected of financial services infrastructure. This approach directly mitigates the primary risks of data leakage, unauthorized access, and non-compliance.

Question 2

A SaaS provider experiences inconsistent application performance due to noisy neighbor effects in a virtualized environment. Analyze the root cause and propose resource allocation and hypervisor-level solutions.

Understanding of the Scenario

The SaaS provider hosts multiple tenants' applications as VMs or containers on shared physical servers. Performance for one tenant degrades unpredictably because another co-located tenant is consuming a disproportionate share of shared resources such as CPU cycles, memory bandwidth, disk I/O, or network bandwidth. This is the classic 'noisy neighbor' problem in multi-tenant virtualized/cloud environments.

Root Cause Analysis

- **Resource Over-commitment:** The hypervisor over-provisions CPU/memory across VMs without enforcing hard limits, so a single high-demand VM can starve others during peak load.
- **Lack of I/O Prioritization:** Shared storage and network paths are not throttled per tenant, allowing one VM's heavy disk or network usage to saturate the shared bus.
- **No CPU/NUMA Awareness:** VMs are scheduled without regard to NUMA node boundaries, causing memory-access latency and cache contention between VMs on the same socket.
- **Absence of Quality-of-Service (QoS) Policies:** No differentiated service levels exist between tenants, so critical and non-critical workloads compete equally for the same resource pool.

Proposed Resource Allocation Solutions

- **Reservations, Limits and Shares:** Configure minimum guaranteed resources (reservations), maximum caps (limits), and proportional priority (shares) for CPU and memory per VM so no tenant can monopolize the host.
- **Right-Sizing and Resource Pools:** Group VMs into resource pools by workload criticality (e.g., premium vs. standard SaaS tiers) and allocate dedicated capacity to premium pools.
- **Dedicated Hosts for Critical Tenants:** Offer high-value or performance-sensitive tenants dedicated physical hosts or dedicated instance types to fully avoid contention.

Proposed Hypervisor-Level Solutions

- **CPU Pinning and NUMA-Aware Scheduling:** Pin VMs to specific cores/sockets and align vCPU/memory placement with NUMA nodes to reduce cross-node contention and cache thrashing.
- **I/O and Network Throttling:** Apply hypervisor-level Storage I/O Control and Network I/O Control (e.g., VMware SIOC/NIOC, KVM cgroups/blkio) to cap and prioritize per-VM I/O bandwidth.
- **Dynamic Resource Scheduling (DRS):** Enable automated load balancing/live migration (e.g., VMware DRS) to move VMs away from congested hosts in real time.
- **Monitoring and Alerting:** Use hypervisor telemetry (esxtop, virt-top, cloud monitoring dashboards) to detect noisy-neighbor patterns early and trigger automated remediation.

Decision Justification

Combining resource-allocation policies (reservations/limits/shares, dedicated hosts) with hypervisor-level controls (CPU pinning, NUMA awareness, I/O throttling, DRS) addresses the root cause directly — uncontrolled resource contention — while preserving the cost efficiency of multi-tenancy for less demanding workloads.

Question 3

An organization wants to migrate legacy monolithic applications into a Service-Oriented Architecture (SOA) in the cloud. Explain the migration strategy and how SOA improves scalability and maintainability.

Understanding of the Scenario

The organization currently runs a tightly coupled monolithic application where all modules (UI, business logic, data access) are bundled into a single deployable unit. This limits independent scaling, slows down releases, and increases the risk of a single failure affecting the entire system. Migrating to SOA in the cloud decomposes the monolith into loosely coupled, reusable services that communicate over standard protocols.

Migration Strategy

- **1. Assessment and Planning:** Analyze the monolith to identify functional boundaries (e.g., billing, inventory, user management) and dependencies, and define the target service catalog.
- **2. Strangler Fig Pattern:** Incrementally extract individual modules into standalone services while routing traffic for migrated functionality through an API gateway, keeping the legacy system operational until all modules are migrated.
- **3. Service Design and Contracts:** Define well-documented service interfaces (WSDL/REST/OpenAPI) and standard message formats (XML/JSON) so services can interoperate regardless of underlying technology.
- **4. Database Decomposition:** Split the shared monolithic database into service-specific data stores to eliminate tight coupling at the data layer, using data-sync or event-driven mechanisms where needed.
- **5. Enterprise Service Bus / API Gateway:** Introduce a middleware layer (ESB or API gateway) to handle service discovery, routing, protocol translation, and cross-cutting concerns like authentication and logging.
- **6. Containerization and Cloud Deployment:** Package each service as an independently deployable unit (e.g., container/VM) and deploy on cloud infrastructure with orchestration for elastic scaling.
- **7. Testing and Cutover:** Perform integration and regression testing for each migrated service, then progressively cut over traffic and decommission the corresponding legacy module.

How SOA Improves Scalability

- Individual services can be scaled independently based on demand (e.g., scaling only the payment service during peak sales) rather than scaling the entire monolith.
- Cloud elasticity (auto-scaling groups, load balancers) can be applied per service, optimizing resource usage and cost.
- Loose coupling allows services to be distributed across multiple servers or regions, improving throughput and fault tolerance.

How SOA Improves Maintainability

- Services can be updated, tested, and redeployed independently without redeploying the entire application, reducing release risk.
- Standardized interfaces promote reusability of services across multiple applications, reducing duplicated code.
- Faults are isolated to individual services, simplifying debugging and improving overall system resilience.

Decision Justification

An incremental strangler-based migration minimizes business disruption while progressively realizing SOA's benefits of independent scalability, fault isolation, and easier long-term maintenance compared to the rigid monolithic design.

Question 4

A government agency is implementing identity federation across multiple departments using cloud services. Explain how identity management, authentication, and privacy policies should be designed to ensure secure access.

Understanding of the Scenario

Multiple government departments, each with their own user directories, need to allow employees and citizens to access shared cloud services using a single identity, without each department having to manage every other department's credentials. This requires an identity federation architecture that establishes trust between Identity Providers (IdPs) and Service Providers (SPs).

Identity Management Design

- **Centralized/Federated Identity Provider:** Establish a central IdP (or federation hub) that each department's existing directory (e.g., LDAP/Active Directory) trusts and federates with, avoiding duplication of identity data.
- **Standard Federation Protocols:** Use SAML 2.0, OAuth 2.0, or OpenID Connect to exchange identity assertions and tokens securely between departments and cloud service providers.
- **Single Sign-On (SSO):** Implement SSO so users authenticate once at their home department's IdP and gain access to authorized services across departments without repeated logins.
- **Attribute-Based Access Control (ABAC):** Grant access based on user attributes (department, role, clearance level) rather than static roles alone, enabling fine-grained, context-aware authorization.

Authentication Design

- **Multi-Factor Authentication (MFA):** Require MFA (e.g., OTP, smart card, biometrics) for sensitive government services to reduce the risk of credential compromise.
- **Token-Based Security:** Use short-lived, digitally signed tokens (JWT/SAML assertions) with encryption in transit (TLS) to prevent token interception or replay attacks.
- **Mutual Trust and Certificate Management:** Establish trust relationships via X.509 certificates between IdPs and SPs, with regular certificate rotation and revocation checks.

Privacy Policy Design

- **Data Minimization:** Share only the minimum set of identity attributes required by the requesting service (e.g., role and clearance, not full personal records).
- **Consent and Purpose Limitation:** Ensure citizen data is used only for the stated purpose of the transaction, with clear consent mechanisms where citizen-facing services are involved.
- **Audit Trails and Accountability:** Maintain immutable logs of authentication and data-access events across departments to support compliance audits and incident investigation.
- **Regulatory Compliance:** Align the design with applicable data-protection laws/frameworks (e.g., national data protection acts, GDPR-equivalent standards) governing government data handling.

Decision Justification

A standards-based federated identity model (SAML/OAuth2/OpenID Connect) with SSO, MFA, and ABAC balances usability for cross-department access with strong authentication and privacy safeguards, ensuring secure and compliant access to shared cloud services.

Question 5

A cloud provider wants to implement real-time user presence detection for collaborative applications. Discuss how presence services can be integrated into cloud architecture while maintaining user privacy and system scalability.

Understanding of the Scenario

Collaborative applications (chat, shared documents, video conferencing) need to show users' real-time status (online, away, typing, in-call) so that other participants can interact effectively. This presence information

changes frequently and must be propagated to potentially thousands of concurrent users with low latency, while ensuring the underlying tracking does not violate user privacy.

Presence Service Architecture

- **Presence Microservice:** Implement presence as an independent microservice, decoupled from core application logic, exposing APIs for status updates and subscriptions.
- **Publish-Subscribe Messaging:** Use a pub/sub messaging system (e.g., MQTT, WebSockets, or a message broker such as Kafka/Redis Pub-Sub) so presence changes are pushed instantly to all subscribed clients instead of being polled.
- **In-Memory Data Store:** Store current presence state in a low-latency, in-memory store (e.g., Redis) with short time-to-live (TTL) entries, since presence data is ephemeral and needs sub-second read/write performance.
- **Heartbeat/Keep-Alive Mechanism:** Clients send periodic heartbeats; if a heartbeat is missed within a timeout window, the service automatically marks the user offline.
- **Event-Driven Updates:** Presence changes trigger events consumed by an event bus, allowing other services (notifications, analytics) to react without tight coupling to the presence service.

Ensuring Scalability

- Horizontally scale presence servers behind a load balancer, using sharding/partitioning of users (e.g., by user ID hash) across multiple presence nodes.
- Use edge servers or regional deployment to reduce latency for geographically distributed users.
- Employ connection multiplexing (WebSocket gateways) to efficiently handle large numbers of persistent client connections per server.

Ensuring User Privacy

- **Opt-In Visibility Controls:** Allow users to configure who can see their presence status (everyone, contacts only, no one) rather than exposing status globally by default.
- **Data Minimization and Retention:** Store only current status with short TTL rather than a persistent history of user activity, and avoid logging fine-grained location/activity details unless required.
- **Encryption and Access Control:** Encrypt presence data in transit (TLS/WSS) and restrict access to presence APIs through authentication and authorization checks.
- **Anonymization for Analytics:** When presence data is aggregated for analytics (e.g., peak usage times), anonymize or aggregate it so individual users cannot be identified.

Decision Justification

A decoupled, pub/sub-based presence microservice backed by an in-memory store provides the low-latency, horizontally scalable foundation needed for real-time collaboration, while opt-in visibility controls and data minimization ensure the design respects user privacy without compromising responsiveness.